

CHAPITRE 2 : FONCTIONS EN C++

Introduction

- ❑ Lorsqu'on a un ensemble de lignes de code qui doivent être exécutées à différents endroits dans un programme, au lieu de réécrire les mêmes lignes de code, il est intéressant de créer des fonctions.
- ❑ Au lieu d'écrire une fonction `main()` de 500 lignes, il est préférable de créer 25 fonctions de 20 lignes
 - ❖ on structure le programme.
 - ❖ il est plus facile de tester chaque fonction.
- ❑ La fonction est une portion de programme que l'on définit à un endroit d'un programme afin de la réutiliser plusieurs fois à différents endroits du code, en évitant la duplication du code.
- ❑ Notons qu'il ne faut jamais dupliquer de code car cela rend le programme, inutilement long, difficile à comprendre et difficile à maintenir.

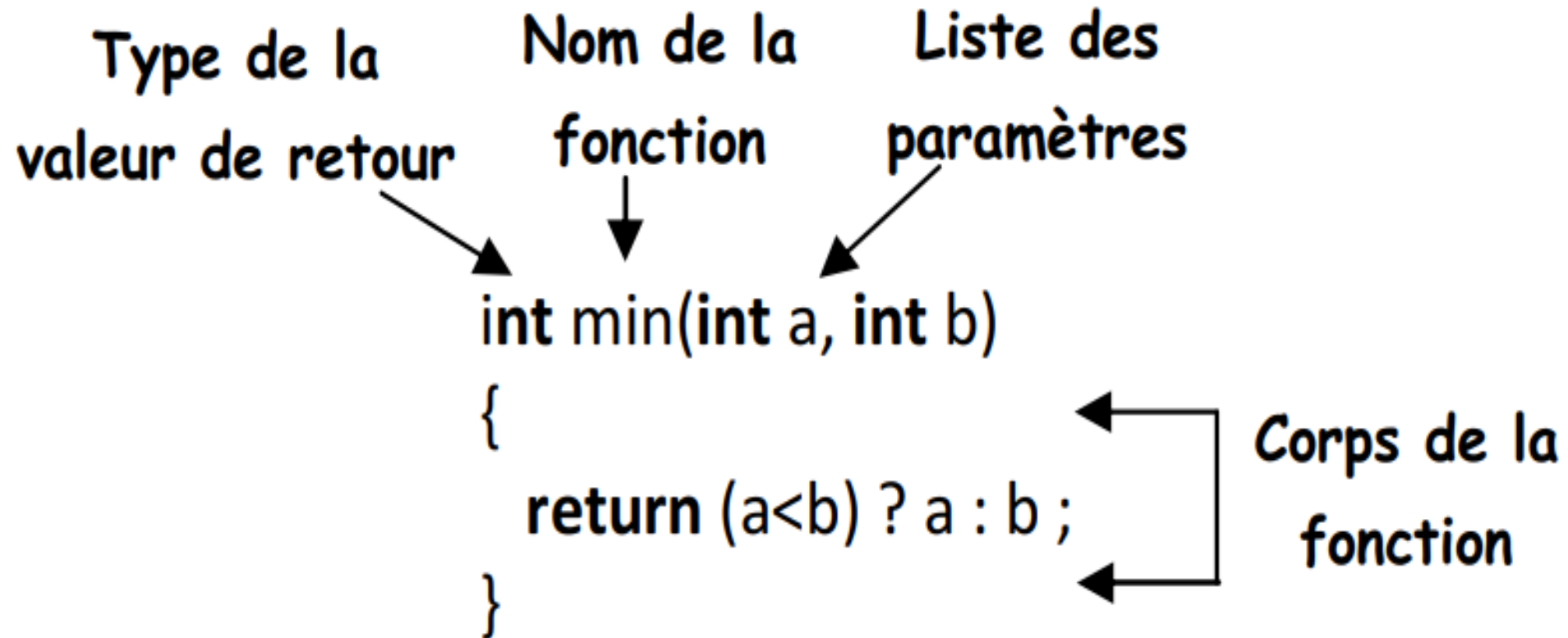
Création et utilisation d'une fonction

Caractérisation d'une fonction

- ❑ Pour utiliser une fonction, on identifie le code à y mettre, on l'extrait dans un endroit du programme et l'on remplace la partie de code extraite par l'appel à la fonction. La fonction reçoit des valeurs d'entrée nécessaires pour et fournit une valeur au reste du programme.
- ❑ Une fonction est caractérisée par :
 - ❖ **un corps**: la portion de programme réutilisée ou à mettre en évidence ;
 - ❖ **un nom**: l'identificateur qui permet de faire référence à cette fonction ;
 - ❖ **des paramètres**: l'ensemble des variables extérieures dont le corps a besoin pour fonctionner ;
 - ❖ **un type et une valeur de retour**: la valeur que la fonction fournit au reste du programme.
- ❑ L'utilisation d'une fonction dans le reste du programme se nomme un « appel à la fonction ».

Création et utilisation d'une fonction

Caractérisation d'une fonction



Création et utilisation d'une fonction

Trois facettes d'une fonction

□ Une fonction a trois facettes :

- ❖ Le **prototype** est un résumé de ce que doit faire la fonction. Il contient le nom, les paramètres qui sont les valeurs nécessaires pour le fonctionnement de la fonction, et le type de la valeur de retour de la fonction.
- ❖ La **définition** contient le prototype et le corps de la fonction, qui est le code exécuté lors de l'utilisation de la fonction.
- ❖ L'**appel** correspond à l'utilisation de la fonction en lui donnant des valeurs effectives pour ses paramètres. La fonction fournit une valeur que l'on utilise dans une expression.

appel

```
z = f(2*u, v+3);
```

prototype

```
double f(double x,  
         double y);
```

définition

```
double f(double a, double b)  
{  
    ...  
}
```

Création et utilisation d'une fonction

- ❑ En pratique le programmeur-concepteur, qui écrit la définition de la fonction, n'est pas forcément la même personne que le programmeur-utilisateur, qui utilise la fonction.
- ❑ Le programmeur-utilisateur a simplement besoin de connaître le prototype de la fonction pour l'utiliser, sans connaître son corps : le prototype sert donc d'accord entre le programmeur-utilisateur et le programmeur-concepteur.

Création et utilisation d'une fonction

Déclaration d'une fonction

- ❑ Toute fonction doit être déclarée avant d'être appelée pour la première fois.
- ❑ Il peut se trouver des situations où une fonction doit être appelée dans une autre fonction définie avant elle. Comme cette fonction n'est pas définie au moment de l'appel, elle doit être déclarée.
- ❑ Le rôle des déclarations est donc de signaler l'existence des fonctions aux compilateurs afin de les utiliser, tout en reportant leur définition de ces fonctions plus loin ou dans un autre fichier.
- ❑ Syntaxe :

type nomDeLa fonction (paramètres) ;

Création et utilisation d'une fonction

Déclaration d'une fonction

❑ Exemple :

double Moyenne(double x, double y); // Fonction avec paramètres et type retour

char LireCaractere(); // Fonction avec type de retour et sans paramètres

void AfficherValeurs(int nombre, double valeur); //Fonction avec paramètres et sans type de retour

Création et utilisation d'une fonction

Définition d'une fonction

❑ Toute fonction doit être définie avant d'être utilisée.

❑ Syntaxe:

```
type nomDeLa fonction (paramètres)
```

```
{
```

```
    Instructions ;
```

```
}
```

❑ Il est possible de créer plusieurs fonctions ayant le même nom. Il faut alors que la liste des arguments des deux fonctions soit différente. C'est ce qu'on appelle la surcharge d'une fonction.

Création et utilisation d'une fonction

Définition d'une fonction

Exemple:

```
#include <iostream>
using namespace std;
double moyenne(double a, double b); // déclaration de la fonction somme :
int main(){
    double x, y, resultat;
    cout << "Tapez la valeur de x : "; cin >> x;
    cout << "Tapez la valeur de y : "; cin >> y;
    resultat = moyenne(x, y); //appel de notre fonction somme
    cout << " Moyenne entre " << x << " et " << y << " est : " << resultat << endl;
    return 0;
}
// définition de la fonction moyenne :
double moyenne(double a, double b){
    double moy;
    moy = (a + b)/2;
    return moy;
}
```

```
Tapez la valeur de x : 5
Tapez la valeur de y : 7.6
Moyenne entre 5 et 7.6 est : 6.3
```

Création et utilisation d'une fonction

Définition d'une fonction

- ❑ Dans ce programme, on a créé une fonction nommée moyenne qui reçoit deux nombres réels a et b en paramètre et qui, une fois qu'elle a terminé, renvoie un autre nombre moy réel qui représente la moyenne de a et b. l'appel de cette fonction se fait au niveau du programme principale (main).
- ❑ Dans cet exemple, le prototype est nécessaire, car la fonction est définie après la fonction main() qui l'utilise. Si le prototype est omis, le compilateur signale une erreur.
- ❑ Le prototype peut être encore écrit de la manière suivante: double moyenne(double , double)

Création et utilisation d'une fonction

Fonctions sans retour

❑ C++ permet de créer des fonctions qui ne renvoient aucun résultat. Mais, quand on la déclare, il faut quand même indiquer un type. On utilise le type **void**. Cela veut tout dire: il n'y a vraiment rien qui soit renvoyé par la fonction.

❑ Exemple:

```
#include <iostream>
using namespace std;
void presenterPgm () ;
int main (){
    presenterPgm();
    return 0;
}
void presenterPgm (){
    cout << "ce pgm ...";
}
```

Création et utilisation d'une fonction

Surcharge des fonctions

- ❑ En C++, Plusieurs fonctions peuvent porter le même nom si leurs signatures diffèrent. La signature d'une fonction correspond aux caractéristiques de ses paramètres
 - ❖ leur nombre
 - ❖ le type respectif de chacun d'eux
- ❑ Le compilateur choisira la fonction à utiliser selon les paramètres effectifs par rapport aux paramètres formels des fonctions candidates.
- ❑ Par exemple, on ne peut pas avoir deux fonctions **int f(int)** et **double f(int)**.

Création et utilisation d'une fonction

Surcharge des fonctions

Exemple:

```
#include <iostream>
```

```
using namespace std;
```

```
// définition de la fonction somme qui calcul la somme de deux réels
```

```
double somme(double a, double b){
```

```
    double r;
```

```
    r = a + b;
```

```
    return r;
```

```
}
```

```
// définition de la fonction somme qui calcul la somme de deux entiers
```

```
double somme(int a, int b){
```

```
    double r;
```

```
    r = a + b;
```

```
    return r;
```

```
}
```

```
// définition de la fonction somme qui calcul la somme de trois réels
```

```
double somme(double a, double b, double c){
```

```
    double r;
```

```
    r = a + b + c; return r;
```

```
}
```

Création et utilisation d'une fonction

```
int main(){
    double x, y, z, resultat;
    cout << "Tapez la valeur de x : "; cin >> x; cout << "Tapez la valeur de y : "; cin >> y;
    cout << "Tapez la valeur de z : "; cin >> z;
    //appel de notre fonction somme (double,double)
    resultat = somme(x, y);
    cout << x << " + " << y << " = " << resultat << endl;
    //appel de notre fonction somme (int,int)
    resultat = somme(static_cast<int>(x), static_cast<int>( y));
    cout << static_cast<int>(x) << " + " << static_cast<int>( y) << " = " << resultat << endl;
    //appel de notre fonction somme (double, double, double)
    resultat = somme(x, y,z);
    cout<< x << " + " << y << " + " << z<< " = " << resultat << endl;
    //appel de notre fonction somme (double, double, double)
    resultat = somme(static_cast<int>(x), static_cast<int>( y), static_cast<int>(z));
    cout << static_cast<int>( x) << " + " << static_cast<int>(y) << " + " << static_cast<int>( z) << "
    = " << resultat << endl;
    return 0;
}
```

```
Tapez la valeur de x : 5.9
Tapez la valeur de y : 6.2
Tapez la valeur de z : 7.13
5.9 + 6.2 = 12.1
5 + 6 = 11
5.9 + 6.2 + 7.13 = 19.23
5 + 6 + 7 = 18
```

Création et utilisation d'une fonction

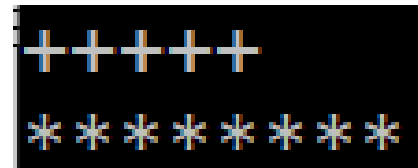
Arguments par défaut

- ❑ On peut, lors de la déclaration d'une fonction, donner des valeurs par défaut à certains paramètres des fonctions. Ainsi, lorsqu'on appelle une fonction, on ne sera pas obligé d'indiquer à chaque fois tous les paramètres!
- ❑ La syntaxe d'un paramètre avec une valeur par défaut est la suivante :

type identificateur = valeur

- ❑ Exemple :

```
#include <iostream>
using namespace std;
void afficheLigne(const char c, const int n=5){
    for(int i(0) ; i<n ; ++i)
        cout << c ;
}
int main(){
    afficheLigne('+') ;
    cout<<endl ;
    afficheLigne('*',8) ;
    return 0 ;}
```



```
+++++
*****
```


Création et utilisation d'une fonction

Arguments par défaut

Il y a quelques règles que vous devez retenir pour les valeurs par défaut:

- ❑ Seul le prototype doit contenir les valeurs par défaut.
- ❑ Les valeurs par défaut doivent se trouver à la fin de la liste des paramètres.
- ❑ Vous pouvez rendre tous les paramètres de votre fonction facultatifs.

```
void f(int i, char c = 'a', double x = 0.0);
```

`f(1)` → correct (vaut `f(1, 'a', 0.0)`)

`f(1, 'b')` → correct (vaut `f(1, 'b', 0.0)`)

`f(1, 3.0)` → **incorrect !**

`f(1, , 3.0)` → **incorrect !**

`f(1, 'b', 3.0)` → correct

Création et utilisation d'une fonction

Passage des paramètres par valeur et par variable

Il y a deux méthodes pour passer des variables en paramètres dans une fonction: le passage par valeur et le passage par variable.

Passage par valeur

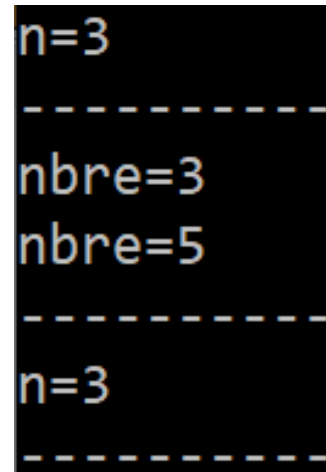
- ❑ La valeur de l'expression passée en paramètre est copiée dans une variable locale. C'est cette variable qui est utilisée pour faire les calculs dans la fonction appelée.
- ❑ Pour mieux comprendre, Prenons le programme suivant qui ajoute 2 à l'argument fourni en paramètre.

Création et utilisation d'une fonction

Passage par valeur

Exemple :

```
#include <iostream>
using namespace std;
void sp (int );
int main(){
    int n = 3;
    cout << "n=" << n << endl;
    sp (n);
    cout << "n=" << n;
    return 0 ;
}
void sp (int nbre){
    cout << "-----" << endl ;
    cout << "nbre=" << nbre << endl;
    nbre = nbre + 2;
    cout << "nbre=" << nbre << endl ;
    cout << "-----"<<endl ;
}
```



```
n=3
-----
nbre=3
nbre=5
-----
n=3
-----
```

Création et utilisation d'une fonction

Passage par variable

Consiste à passer l'adresse d'une variable en paramètre. Toute modification du paramètre dans la fonction affecte directement la variable passée en argument correspondant, puisque la fonction accède à l'emplacement mémoire de son argument. Il existe 2 possibilités pour transmettre des paramètres par variables :

- ❑ Les références

- ❑ Les pointeurs

Passage par références

Consiste à passer une référence de la variable en argument. Ainsi, aucune variable temporaire ne sera créée par la fonction et toutes les opérations (de la fonction) seront effectuées directement sur la variable. Le plus simple est d'utiliser le mécanisme de référence « & ».

Création et utilisation d'une fonction

Passage par références

Exemple :

```
#include <iostream>
using namespace std;
void sp (int &); // ajout de la référence au niveau du prototype
int main(){
    int n = 3;
    cout << "n=" << n << endl;
    sp (n); //appel de la fonction
    cout << "n=" << n << endl ;;
    return 0 ;
}
void sp (int & nbre) //Ajout de la référence dans la fonction
{ cout << "-----" << endl ;
  cout << "nbre=" << nbre << endl;
  nbre = nbre + 2;
  cout << "nbre=" << nbre << endl ;
  cout << "-----" << endl ;
}
```

```
n=3
-----
nbre=3
nbre=5
-----
n=5
```

Création et utilisation d'une fonction

Passage par adresses (pointeurs)

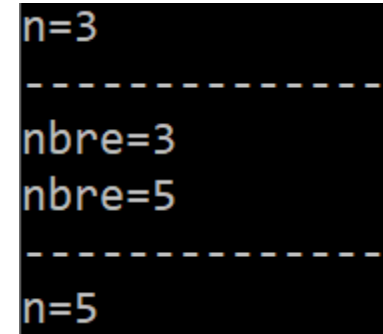
- ❑ Consiste à passer l'adresse d'une variable en paramètre.
- ❑ Toute modification du paramètre dans la fonction affecte directement la variable passée en argument correspondant, puisque la fonction accède à l'emplacement mémoire de son argument.
- ❑ Le pointeur indique au compilateur que ce n'est pas la valeur qui est transmise, mais une adresse (un pointeur).

Création et utilisation d'une fonction

Passage par adresses (pointeurs)

Exemple :

```
#include <iostream>
using namespace std;
void sp (int *); // ajout de l'étoile au niveau du prototype
int main(){
    int n = 3;
    cout << "n=" << n << endl;
    sp (&n); //appel de la fonction
    cout << "n=" << n<<endl ;
    return 0 ;
}
void sp (int * nbre) //Ajout de la référence dans la fonction
{
    cout << "-----"<<endl ;
    cout << "nbre=" << *nbre << endl;
    *nbre = *nbre + 2;
    cout << "nbre=" <<* nbre<<endl ;
    cout << "-----"<<endl ;
}
```



```
n=3
-----
nbre=3
nbre=5
-----
n=5
```

Création et utilisation d'une fonction

Fonctions inline

❑ Parfois le temps d'exécution d'une fonction est petit comparé au temps nécessaire pour appeler la fonction. Le mot clé inline informe le compilateur qu'un appel à cette fonction peut être remplacé par le corps de la fonction.

❑ Syntaxe : **inline** type fonct(liste_des_arguments){...}

❑ Exemple

```
inline int max(int a, int b){  
    return (a < b) ? b : a;  
}  
  
int main(){  
    int m = max(134, 876);  
    cout<< "m=" << m<<endl ; return 0;  
}
```

❑ Les fonctions "inline" permettent de gagner au niveau de temps d'exécution, mais augmente la taille des programmes en mémoire.

Inférence de type

- ❑ L'inférence de type fait référence à la déduction automatique du type de données d'une expression dans un langage de programmation.
- ❑ Avant C++ 11, chaque type de données devait être explicitement déclaré au moment de la compilation, limitant les valeurs d'une expression à l'exécution, mais après la nouvelle version de C++, de nombreux mots-clés sont inclus, ce qui permet à un programmeur de laisser la déduction de type au compilateur lui-même .
- ❑ Vous utilisez le mot clé auto pour indiquer que le compilateur doit déduire le type d'une variable.
- ❑ Exemple 1
 - auto m (10); // m est de type int**
 - auto n (5.7); // n est de type float**
 - auto p (« bonjour»); // p est de type string**
- ❑ Le compilateur déduira les types de m, n et p à partir des valeurs initiales que vous fournissez.

Les fonctions en C++11

❑ En C++11, l'une des fonctionnalités majeures introduites est la possibilité de spécifier le type de retour des fonctions en utilisant le mot-clé **auto**. Cela permet d'inférer automatiquement le type de retour en fonction de ce que la fonction retourne. Voici un exemple de changement de déclaration de fonction en utilisant **auto** :

❑ Avant C++11 :

```
int additionner(int a, int b) {  
    return a + b;  
}
```

❑ Avec C++11, vous pouvez écrire la même fonction de cette manière :

```
auto additionner(int a, int b) -> int {  
    return a + b;  
}
```

❑ Dans cet exemple, **auto** est utilisé pour indiquer que le type de retour sera inféré automatiquement, suivi de **-> int** pour préciser que la fonction retourne un entier.

❑ Cela simplifie la déclaration de fonctions, en particulier lorsque le type de retour est complexe ou difficile à écrire explicitement.

Les fonctions en C++11

```
#include <iostream>
// Fonction qui additionne deux entiers et retourne le résultat
auto additionner(int a, int b) -> int {
    return a + b;
}
int main() {
    int x = 5;
    int y = 3;
    // Appel de la fonction additionner
    auto resultat = additionner(x, y);
    // Affichage du résultat
    std::cout << "La somme de " << x << " et " << y << " est : " << resultat << std::endl;
    return 0;
}
```

❑ Dans cet exemple, nous avons la fonction `additionner` qui prend deux entiers en entrée et retourne leur somme. Au lieu de spécifier explicitement **int** comme type de retour, nous utilisons `auto` suivi de `-> int` pour indiquer que la fonction retourne un entier.

Les fonctions lambda en c++

- ❑ En C++, nous pouvons utiliser des fonctions sans nom (fonctions lambda). En fait, une telle fonction est un objet d'un type spécial, et cet objet peut être affecté à une variable. Après cela, nous pouvons appeler la variable comme s'il s'agissait d'une fonction.
- ❑ Ainsi, une fonction lambda est une construction sémantique qui détermine une fonction. La syntaxe d'une fonction lambda est la suivante :
 - ❑ Syntaxe

```
[](parametres)->return_type{  
    // instructions  
}
```
- ❑ Les fonctions lambda sont utiles pour des opérations locales et ponctuelles, et elles offrent une syntaxe concise et lisible. De plus, vous pouvez capturer des variables externes en les spécifiant dans les crochets [], ce qui permet à la fonction lambda d'accéder à ces variables.

Les fonctions lambda en c++

❑ Example 1

```
#include <iostream>
using namespace std;
int main() {
    auto somme=[](int n)->int{
        int s=0;
        for(int k=1;k<=n;k++){ s+=k; }
        return s;
    };
    cout << "La somme de 1 a " << 7 << " = " << somme(7) << endl;
    auto fact=[](int n){
        int f=1;
        for(int k=1;k<=n;k++){ f*=k; }
        return f;
    };
    cout << "La factorielle de 7 = " << fact(7);
    return 0;
}
```

Les fonctions lambda en c++

❑ Example 2

```
#include <iostream>
using namespace std;
int main() {
    // Définition d'une fonction lambda qui additionne deux entiers
    auto addition = [](int a, int b)->int {
        return a + b;
    };
    int x(5), y(3);
    // Appel de la fonction lambda
    int resultat = addition(x, y);
    cout << "La somme de " << x << " et " << y << " est : " << resultat << endl;
    return 0;
}
```

La boucle for basée sur les collections

- ❑ La boucle for basée sur une collection itère sur toutes les valeurs de la collection donnée.
- ❑ Cela soulève une question immédiate : qu'est-ce qu'une collection ? Une collection est un ensemble d'éléments, une chaîne de caractères est une collection de caractères.
- ❑ C++11 a introduit une syntaxe de boucle for basée sur les collections pour l'itération dans les tableaux et autres types de conteneurs. À chaque itération, l'élément suivant du tableau est lié à la variable spécifiée, dans ce cas une variable **coll_declaration**, et la boucle continue jusqu'à ce qu'elle ait parcouru tout le tableau.
- ❑ Syntaxe
for (coll_declaration : coll_expression)
 corps de la boucle;

La boucle for basée sur les collections

❑ Example 1

```
#include <iostream>
using namespace std;
int main(void){
    int T[4] = {1, 7, 13, 17};
    for (int elem : T) {
        cout << elem << '\t';
    }
    return 0;
}
```

❑ Example 2

```
#include <iostream>
using namespace std;
int main(void){
    int T[4] = {1, 7, 13, 17};
    for (auto elem : T) {
        cout << elem << '\t';
    }
    return 0;
}
```

1 7 13 17